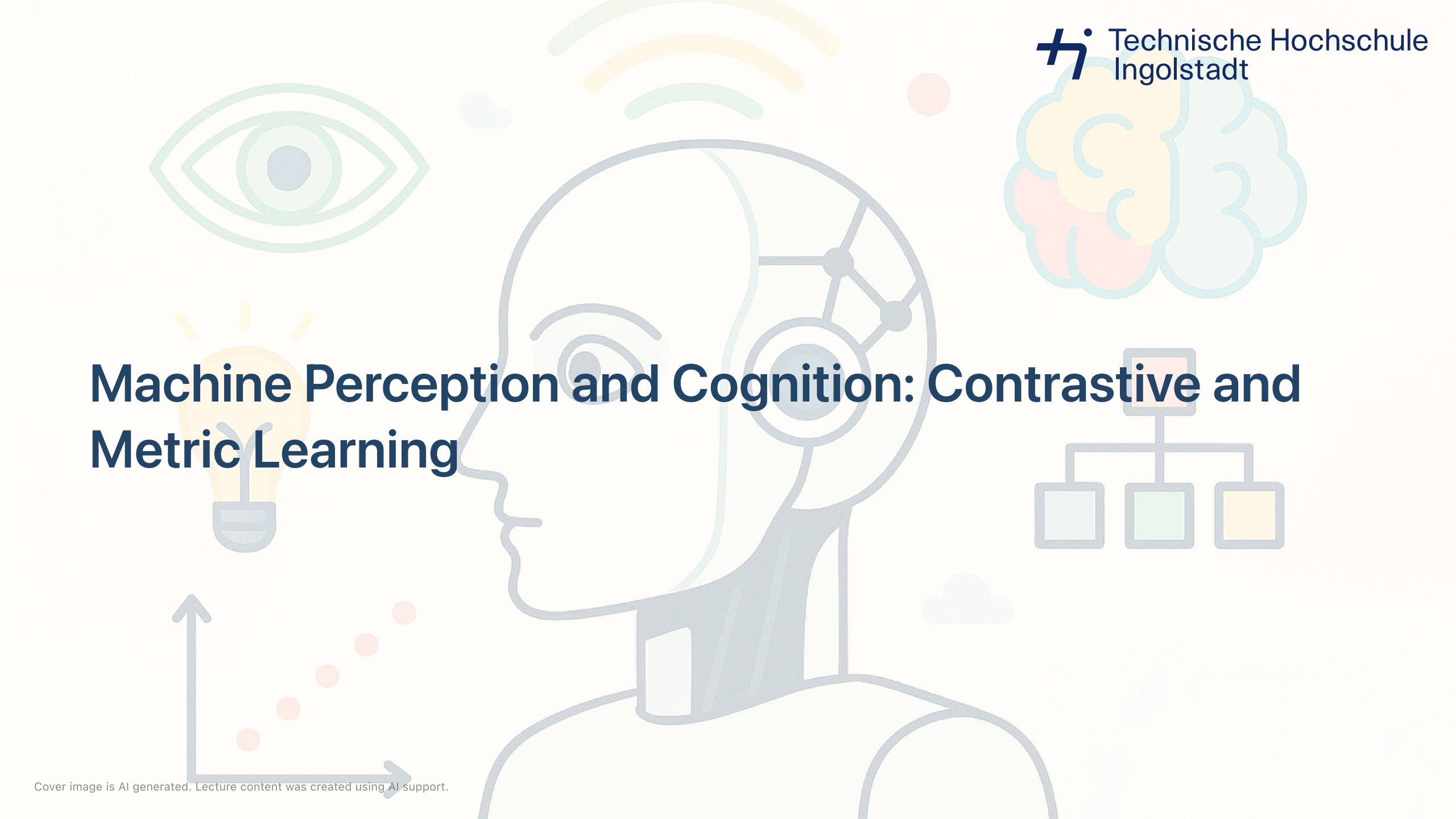
Technische Hochschule Ingolstadt



Machine Perception and Cognition: Contrastive and Metric Learning

Cover image is AI generated. Lecture content was created using AI support.

Learning Goals

After this lecture you will be able to:

- Explain why standard classification fails for **open-set recognition** tasks such as face verification
- Describe the **embedding space** concept and state what a good metric learning model should achieve
- Formulate the **contrastive loss** and the **triplet loss** and explain the role of the margin α
- Explain **triplet mining** strategies (easy, hard, semi-hard negatives) and why mining matters

Why Classification Falls Short

Standard classifiers assign inputs to a **fixed set of known classes**:

$$\hat{y} = \arg\max_{k} \ \mathrm{softmax}(\mathbf{W} f_\theta(\mathbf{x}))_k$$

This breaks for many real-world tasks:

Problem	Why a classifier fails
Face verification	New identities appear at runtime — not in training set
Product image search	Millions of products; retraining for each new item is impractical
Medical image retrieval	Need to rank similar cases, not assign a fixed label
Person re-identification	Same person, different cameras and clothing

➡️ We need a model that measures **similarity** rather than assigning fixed labels.

The Embedding Space Idea

Key idea: map each input $\mathbf{x}$ to a point $\mathbf{z} = f_\theta(\mathbf{x})$ in a low-dimensional **embedding space** $\mathbb{R}^d$, such that:

$$d(\mathbf{z}_i, \mathbf{z}_j) \text{ is small } \iff \mathbf{x}_i \text{ and } \mathbf{x}_j \text{ are semantically similar}$$

Desired geometry:

- Same class → embeddings cluster **closely together**
- Different class → embeddings are **far apart**
- Distances generalize to **unseen classes** at test time

The Embedding Space Idea

T-SNE embedding of MNIST. Same digits are close:

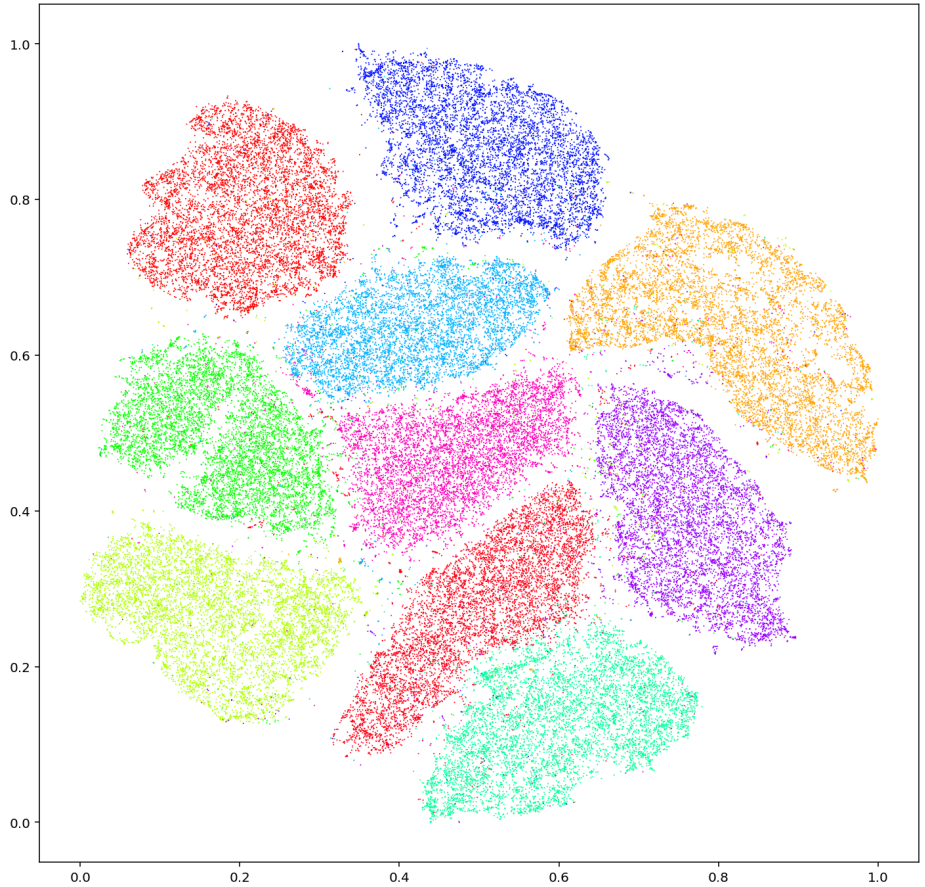


Image source: Kyle McDonald, CC BY 2.0 (https://creativecommons.org/licenses/by/2.5), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:T-SNE_Embedding_of_MNIST.png

Embedding Space

A good embedding space encodes semantic relationships:

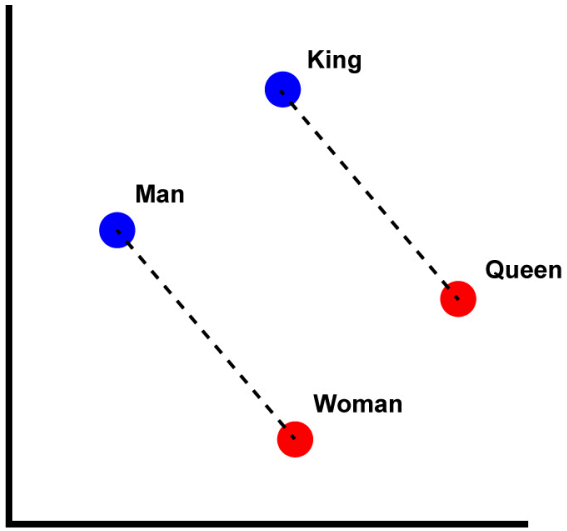


Image source: Singerep, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Word_vector_illustration.jpg

What is Metric Learning?

Metric learning is the task of learning an embedding function f_θ such that the Euclidean distance in embedding space reflects semantic similarity between inputs.

Using the Euclidean distance between embeddings,

$$d_\theta(\mathbf{x}_i, \mathbf{x}_j) = \|f_\theta(\mathbf{x}_i) - f_\theta(\mathbf{x}_j)\|_2$$

the Euclidean distance on the embedding space is a valid metric.

A valid metric on the embedding space must satisfy:

1. **Non-negativity**: $d(\mathbf{z}_i, \mathbf{z}_j) \geq 0$
2. **Identity of indiscernibles**: $d(\mathbf{z}_i, \mathbf{z}_j) = 0 \Leftrightarrow \mathbf{z}_i = \mathbf{z}_j$
3. **Symmetry**: $d(\mathbf{z}_i, \mathbf{z}_j) = d(\mathbf{z}_j, \mathbf{z}_i)$
4. **Triangle inequality**: $d(\mathbf{z}_i, \mathbf{z}_k) \leq d(\mathbf{z}_i, \mathbf{z}_j) + d(\mathbf{z}_j, \mathbf{z}_k)$

What is Metric Learning?

On the original input space, $d_\theta(\mathbf{x}_i, \mathbf{x}_j)$ is generally a **pseudometric**, because two different inputs can map to the same embedding, violating identity of indiscernibles on the input space (for example, two photos of the same person can map to the same embedding).

Distance and Similarity in Embedding Space

Two common choices for comparing embeddings $\mathbf{z}_i, \mathbf{z}_j \in \mathbb{R}^d$ are a **distance measure** and a **similarity measure**:

Euclidean distance:

$$d_E(\mathbf{z}_i, \mathbf{z}_j) = \|\mathbf{z}_i - \mathbf{z}_j\|_2 = \sqrt{\sum_{k=1}^{d} (z_{ik} - z_{jk})^2}$$

Cosine similarity (after L2-normalizing embeddings to unit norm $\|\mathbf{z}\|_2 = 1$):

$$\cos(\mathbf{z}_i, \mathbf{z}_j) = \mathbf{z}_i^\top \mathbf{z}_j \in [-1, 1]$$

Distance and Similarity in Embedding Space

For normalized embeddings $\hat{\mathbf{z}} = \mathbf{z}/\|\mathbf{z}\|_2$, the two are equivalent up to a monotonic transformation:

$$\|\hat{\mathbf{z}}_i - \hat{\mathbf{z}}_j\|_2^2 = 2 - 2\hat{\mathbf{z}}_i^\top \hat{\mathbf{z}}_j$$

Why Cosine Similarity?

Volume of an n-dimensional ball of radius R:

$$V_n(R) = \frac{\pi^{n/2}}{\Gamma\left(\frac{n}{2}+1\right)} R^n$$

Fraction of volume of a smaller ball with radius αR inside a larger ball:

$$\frac{V_n(\alpha R)}{V_n(R)} = \alpha^n$$

Why Cosine Similarity?

Examples for different dimensionalities and $\alpha = 0.9$:

- 3D: 0.729
- 4D: 0.656
- 5D: 0.590
- 512D: $\approx 3.733 \times 10^{-24}$

In high-dimensional balls, most of the volume lies near the boundary, so many points already differ little in radius.

Therefore, a common choice is to remove the radial component entirely with L2 normalization; then cosine similarity compares only direction (equivalently, angle).

Siamese Networks

A **Siamese network** processes two inputs through **shared weights** to produce a pair of embeddings:

$$\mathbf{z}_i = f_\theta(\mathbf{x}_i), \quad \mathbf{z}_j = f_\theta(\mathbf{x}_j)$$

The same network f_θ (identical weights) is applied to both inputs — hence the name *Siamese* (twin).

Training signal: compare the pair output distance against the ground-truth label:

- **Positive pair** $(y = 1)$: same class — distance should be small
- **Negative pair** $(y = 0)$: different class — distance should be large

Contrastive Loss

The **contrastive loss** (Hadsell et al., 2006) trains a Siamese network on pairs:

$$\mathcal{L}_{\text{contrastive}} = \frac{1}{2N} \sum_{i=1}^{N} \left[y_i \, d_i^2 + (1 - y_i) \max(0,\, m - d_i)^2 \right]$$

where:

- $d_i = \| f_\theta(\mathbf{x}_i^{(a)}) - f_\theta(\mathbf{x}_i^{(b)}) \|_2$ — Euclidean distance of the pair
- $y_i \in \{0, 1\}$ — 1 for a positive pair, 0 for a negative pair
- $m > 0$ — **margin**: minimum required distance for negative pairs

Contrastive Loss

Intuition:

- Positive pairs ($y = 1$): minimizes d_i^2 → pull embeddings together
- Negative pairs ($y = 0$): minimizes $\max(0, m - d_i)^2$ → push embeddings apart (only if $d_i < m$)

Pairs: Positive and Negative

Positive pair: two samples from the **same** class

- Same person, different photos → $(x_{\text{face1}}, x_{\text{face2}}),\ y = 1$

Negative pair: two samples from **different** classes

- Two different people → $(x_{\text{person A}}, x_{\text{person B}}),\ y = 0$

Challenge: sampling pairs naively leads to **imbalanced training**:

- With C classes and N total samples, negative pairs outnumber positive pairs by a factor of $\sim C$
- Most negative pairs are **trivially easy** (very far apart already) → little gradient signal

➡️ Pair selection strategy strongly affects training efficiency.

From Pairs to Triplets

Limitation of contrastive loss: it considers only one pair at a time — positive and negative constraints are not enforced simultaneously.

Triplet loss (Schroff et al., 2015) addresses this by training on **triplets**:

$$\left(\mathbf{x}^a, \mathbf{x}^p, \mathbf{x}^n\right)$$

- $\mathbf{x}^a$ — **anchor**: reference sample
- $\mathbf{x}^p$ — **positive**: same class as anchor
- $\mathbf{x}^n$ — **negative**: different class from anchor

Goal: the anchor should be closer to the positive than to the negative:

$$\left\| f_\theta(\mathbf{x}^a) - f_\theta(\mathbf{x}^p) \right\|_2 < \left\| f_\theta(\mathbf{x}^a) - f_\theta(\mathbf{x}^n) \right\|_2$$

Triplet Loss

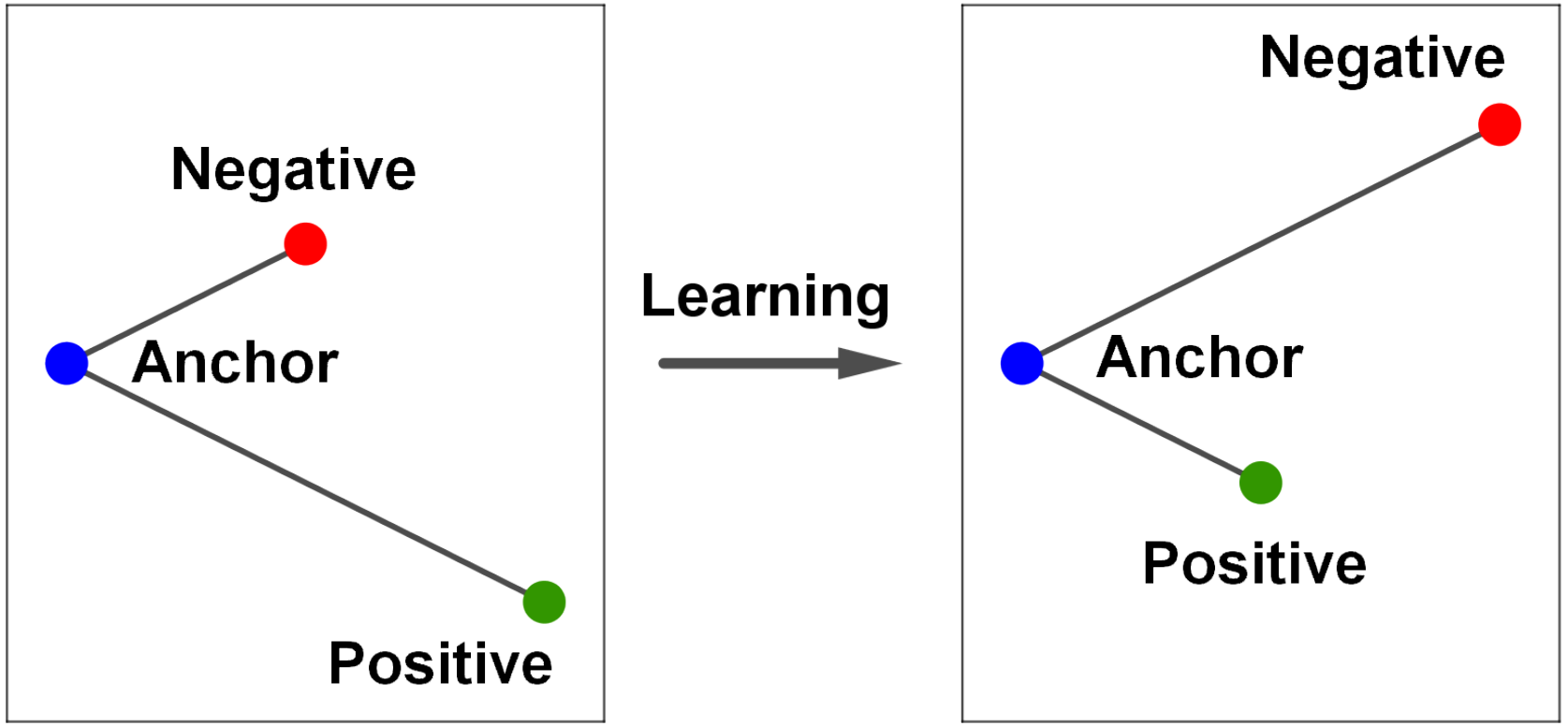


Image source: Krishnachandranvn, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Triplet_Loss_Minimization.png

Triplet Loss

The **triplet loss** enforces a margin α between the positive and negative distances:

$$\mathcal{L}_{\text{triplet}} = \frac{1}{N} \sum_{i=1}^{N} \max\Big(0,\ d_{ap}^{(i)} - d_{an}^{(i)} + \alpha\Big)$$

where:

- $d_{ap}^{(i)} = \| f_\theta(\mathbf{x}^{a(i)}) - f_\theta(\mathbf{x}^{p(i)}) \|_2$ — anchor-to-positive distance
- $d_{an}^{(i)} = \| f_\theta(\mathbf{x}^{a(i)}) - f_\theta(\mathbf{x}^{n(i)}) \|_2$ — anchor-to-negative distance
- $\alpha > 0$ — **margin** (e.g. $\alpha = 0.2$)

Triplet Loss

Effect of margin α:

- Without margin ($\alpha = 0$): the trivial solution $f_\theta(\mathbf{x}) = \mathbf{0}$ satisfies the constraint
- With margin: the negative must be at least α farther away than the positive — forces a well-separated embedding space

Triplet Loss — Geometry

Loss is zero (no gradient) when: $d_{ap} + \alpha \leq d_{an}$ → triplet already satisfied

Loss is positive (gradient signal) when: $d_{ap} + \alpha > d_{an}$ → triplet violated

Training pushes d_{ap} down and d_{an} up until the margin is satisfied for all triplets.

Triplet Mining — The Challenge

With N training samples, the number of possible triplets is $O(N^3)$.

Key insight: not all triplets are equally informative:

Triplet type	Condition	Effect
Easy (satisfied)	$d_{ap} + \alpha \leq d_{an}$	Loss = 0, **no gradient**
Hard negative	$d_{an} < d_{ap}$	Strong gradient, but can **destabilize** training
Semi-hard negative	$d_{ap} < d_{an} < d_{ap} + \alpha$	Loss > 0, **stable gradient** — recommended

➡️ Training on **random triplets** wastes compute on easy ones. **Mining** selects informative triplets.

Online Triplet Mining

Offline mining: select hard triplets before each epoch → expensive and stale.

Online mining (within a mini-batch): for each mini-batch of B samples with class labels:

1. Compute all $O(B^2)$ pairwise distances
2. For each anchor, select a positive and negative according to the chosen strategy
3. Form triplets **on the fly** — fresh and computationally cheap

Batch construction: sample P classes with K samples each (batch size $B = P \times K$):

- All $K(K-1)$ anchor-positive pairs per class are available
- Negatives come from the other $P - 1$ classes

➡️ Online mining with semi-hard negatives is the standard recipe (used in FaceNet, Google Street View).

Offline Triplet Mining

Offline mining constructs triplets **before** an epoch begins, using the model state from the previous epoch:

1. Run a forward pass on the **entire dataset** → compute all N embeddings
2. Enumerate candidate triplets and score them by their current loss value
3. Select the most informative triplets (e.g. hardest $k\%$) and store them
4. Train on the stored triplets for one epoch, then repeat

Offline Triplet Mining

Advantages:

- Can find the **globally hardest** triplets across the full dataset
- Simple to implement as a pre-processing step

Disadvantages:

- Embeddings become **stale** after each weight update — selected triplets may no longer be hard
- Computationally expensive: requires a full forward pass over N samples before each epoch
- Memory-intensive: must hold all N embeddings simultaneously

➡️ Offline mining is rarely used in practice for large-scale training.

Online Triplet Mining — In Detail

Online mining selects triplets **within each mini-batch** during training.

Batch construction: sample P classes with K samples each → batch size $B = P \times K$

For each forward pass:

1. Compute all $B(B-1)$ directed pairwise distances from the current embeddings
2. For each anchor, enumerate all valid positives (same class) and negatives (different class)
3. Apply the mining strategy to select triplets — **no extra forward passes needed**

Online Triplet Mining — In Detail

Why online mining works well:

- Triplets use **current** embeddings → always relevant to the model's present state
- Each batch contains $P \cdot K(K-1)$ anchor-positive pairs in total and negatives from the $P-1$ other classes per anchor
- Effectively generates far more training signal per forward pass than random sampling

Hard Negative Mining

A **hard negative** is a negative sample that is **closer to the anchor than the positive**:

$$d_{an} < d_{ap}$$

Why it matters:

- The embedding space is actively "wrong" — the network ranks a different-class sample as more similar than the same-class positive
- Produces the **largest gradient signal** → the strongest corrective update

Hard Negative Mining

Why pure hard negative mining can fail:

- In early training, embeddings are unreliable → many negatives appear hard, producing noisy or contradictory gradients
- Hard negatives are susceptible to **label noise**: two samples of the same class may be mislabeled as negatives, causing the model to learn incorrect geometry
- Repeatedly selecting only the hardest cases can overemphasize outliers and make optimization **numerically unstable**

➡️ Pure hard negative mining is too aggressive — it requires warm-up strategies or curriculum scheduling.

Semi-Hard Negative Mining

A **semi-hard negative** satisfies:

$$d_{ap} < d_{an} < d_{ap} + \alpha$$

The negative is **farther than the positive** (correct ordering) but **within the margin** (still violates the constraint).

Why semi-hard negatives are preferred:

- Guaranteed non-zero loss → training always makes progress
- The positive-negative ordering is already correct → lower risk of instability than with hardest negatives
- Violations are moderate → typically more stable updates than using only the hardest negatives

Batch-Hard Mining

Batch-hard mining (Hermans et al., 2017) is a widely used online strategy.

For each anchor $\mathbf{x}_i^a$ in the batch, select simultaneously:

- **Hardest positive**: $\mathbf{x}^{p*} = \arg\max_{p:\, y_p = y_i} \left\| f_\theta(\mathbf{x}_i^a) - f_\theta(\mathbf{x}^p) \right\|_2$
- **Hardest negative**: $\mathbf{x}^{n*} = \arg\min_{n:\, y_n \neq y_i} \left\| f_\theta(\mathbf{x}_i^a) - f_\theta(\mathbf{x}^n) \right\|_2$

The batch-hard loss is:

$$\mathcal{L}_{\text{BH}} = \frac{1}{B} \sum_{i=1}^{B} \max\left(0,\ \max_p d_{ap_i} - \min_n d_{an_i} + \alpha\right)$$

Effect: every sample acts as an anchor and the network always sees the worst-case triplet available in the batch — strong, consistent gradient signal without the collapse risk of global hard mining.

Normalized Embeddings and the Unit Hypersphere

L2 normalization maps all embeddings onto the unit hypersphere $\mathcal{S}^{d-1}$:

$$\hat{\mathbf{z}} = \frac{\mathbf{z}}{\|\mathbf{z}\|_2}, \quad \|\hat{\mathbf{z}}\|_2 = 1$$

Why normalize?

- Prevents the network from trivially satisfying the triplet constraint by scaling all embeddings

In code (PyTorch):

```
z = torch.nn.functional.normalize(z, p=2, dim=1)
```

Application: Face Recognition

Face recognition is the canonical application of metric learning in computer vision.

Two sub-tasks:

- **Face verification (1:1)**: "Is this photo the same person as the reference?" → binary decision
- **Face identification (1:N)**: "Who is this person?" → nearest neighbor in a gallery of N enrolled identities

Application: Face Recognition

Why metric learning is the right tool:

- Number of identities can be **millions** (e.g. social media, law enforcement)
- New identities are enrolled **without retraining** — just add their embedding to the gallery
- Works in **open-set** settings: the test identity may never have been seen during training

FaceNet

FaceNet (Schroff et al., 2015) was the first system to apply the triplet loss at scale to face recognition.

Architecture:

- Backbone: Inception-style CNN (later versions use ResNet variants)
- Final layer: L2-normalized embedding of dimension $d = 128$
- Loss: triplet loss with online semi-hard negative mining

Training scale: 100–200 million face images from 8 million identities

Key contribution: showed that the triplet loss with online semi-hard negative mining scales to production-grade face recognition.

Face Verification vs. Identification

Task	Query	Database	Decision
Verification (1:1)	One probe image	One enrolled reference	Same person? Yes / No
Identification (1:N)	One probe image	Gallery of N identities	Who is it? (Top-k candidates)

Verification uses a **threshold** τ on the embedding distance:

$$\text{same person} \iff d(\hat{\mathbf{z}}_{\text{probe}}, \hat{\mathbf{z}}_{\text{reference}}) < \tau$$

Identification uses **nearest neighbor search** in the gallery:

$$\hat{y} = \arg\min_{k=1,\ldots,N} d(\hat{\mathbf{z}}_{\text{probe}}, \hat{\mathbf{z}}_k)$$

Evaluation Metrics for Face Recognition

Threshold τ controls the trade-off between **false accept** and **false reject**:

- **FAR** (False Accept Rate): fraction of impostor pairs accepted as genuine
- **FRR** (False Reject Rate): fraction of genuine pairs rejected

The **ROC curve** plots FAR vs. 1 – FRR as τ varies; **AUC** (area under curve) summarizes it in one number.

Receiver Operating Characteristics (ROC)

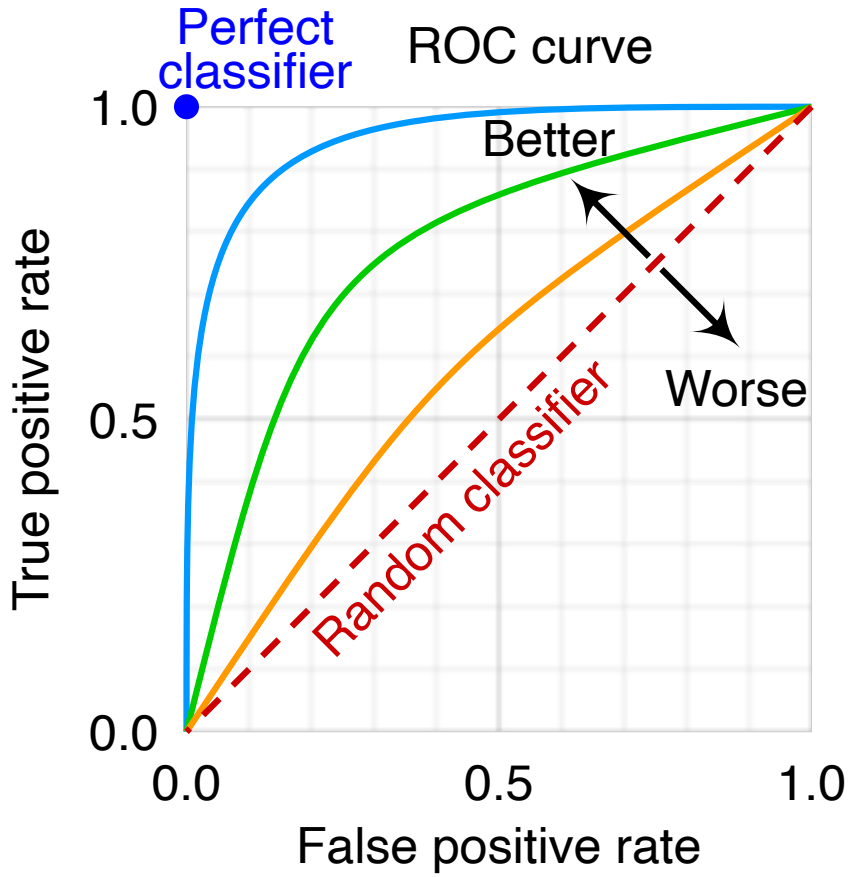


Image source: cmglee, MartinThoma, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Roc_curve.svg

PyTorch: Embedding Network

A minimal embedding network for image inputs:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class EmbeddingNet(nn.Module):
    def __init__(self, backbone, embed_dim=128):
        super().__init__()
        self.backbone = backbone          # e.g. ResNet-50 (pretrained)
        self.head = nn.Linear(backbone.output_dim, embed_dim)

    def forward(self, x):
        features = self.backbone(x)       # (B, D)
        z = self.head(features)           # (B, embed_dim)
        z = F.normalize(z, p=2, dim=1)   # L2 normalize → unit sphere
        return z
```

PyTorch: Triplet Loss

```python
class TripletLoss(nn.Module):
    def __init__(self, margin=0.2):
        super().__init__()
        self.margin = margin

    def forward(self, anchor, positive, negative):
        # anchor, positive, negative: (B, embed_dim), already L2-normalized
        d_ap = (anchor - positive).pow(2).sum(dim=1).sqrt()  # (B,)
        d_an = (anchor - negative).pow(2).sum(dim=1).sqrt()  # (B,)
        loss = F.relu(d_ap - d_an + self.margin)             # (B,)
        return loss.mean()
```

- `F.relu(...)` implements the $\max(0, \cdot)$ hinge
- `.mean()` averages over the mini-batch
- Gradients flow to both the positive (pull) and negative (push) paths

PyTorch: Training Loop with Triplet Loss

```
model = EmbeddingNet(backbone, embed_dim=128)
criterion = TripletLoss(margin=0.2)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)

for epoch in range(num_epochs):
    for anchors, positives, negatives in triplet_loader:
        z_a = model(anchors)
        z_p = model(positives)
        z_n = model(negatives)

        loss = criterion(z_a, z_p, z_n)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Beyond Face Recognition — Other Applications

The same metric learning framework applies across many domains:

- Person re-identification
- Product image search
- Medical image retrieval
- Speaker/writer verification
- Document retrieval

In each case: the **encoder** changes (CNN for images, transformer for text/audio) but the **loss** and **training procedure** remain the same.

k-NN Classification with Embeddings

Once a metric learning model is trained, classification on **new classes** requires no retraining:

1. **Enroll**: embed all labeled reference samples → build a gallery $\{(\hat{\mathbf{z}}_i, y_i)\}$
2. **Query**: embed new sample $\mathbf{x}$ → $\hat{\mathbf{z}}$
3. **Predict**: find k nearest neighbors in the gallery; majority vote → predicted class

This is the basis of **few-shot learning** — the model generalizes from as few as 1 example per class.

Challenges and Pitfalls

Degenerate embeddings / collapse: many inputs map to nearly the same vector:

- Can occur with degenerate objectives (for example triplet loss with $\alpha = 0$) or with poor mining / regularization
- Prevention: use a positive margin, L2 normalization, informative negatives, and monitor embedding variance

Mode collapse in self-supervised learning: representations become constant despite non-trivial inputs

Sensitivity to augmentation: contrastive learning is **only as good as the augmentation policy** — the model learns to be invariant to whatever transforms are applied

Computational cost of triplet mining: naive online mining requires computing an $O(B^2)$ pairwise distance matrix per batch

Recap!

Metric learning fundamentals:

- Standard classifiers cannot handle **open-set** recognition — they assign fixed labels to known classes only
- Metric learning trains an **embedding network** f_θ so that similar inputs are nearby and dissimilar inputs are far apart in $\mathbb{R}^d$
- **Contrastive loss**: trains on pairs — pulls positives together (d_{ap}^2) and pushes negatives apart ($\max(0, m - d_{an})^2$)
- **Triplet loss**: trains on triples $(\mathbf{x}^a, \mathbf{x}^p, \mathbf{x}^n)$ — enforces a margin α so that the negative is at least α farther than the positive: $\max(0, d_{ap} - d_{an} + \alpha)$

Recap!

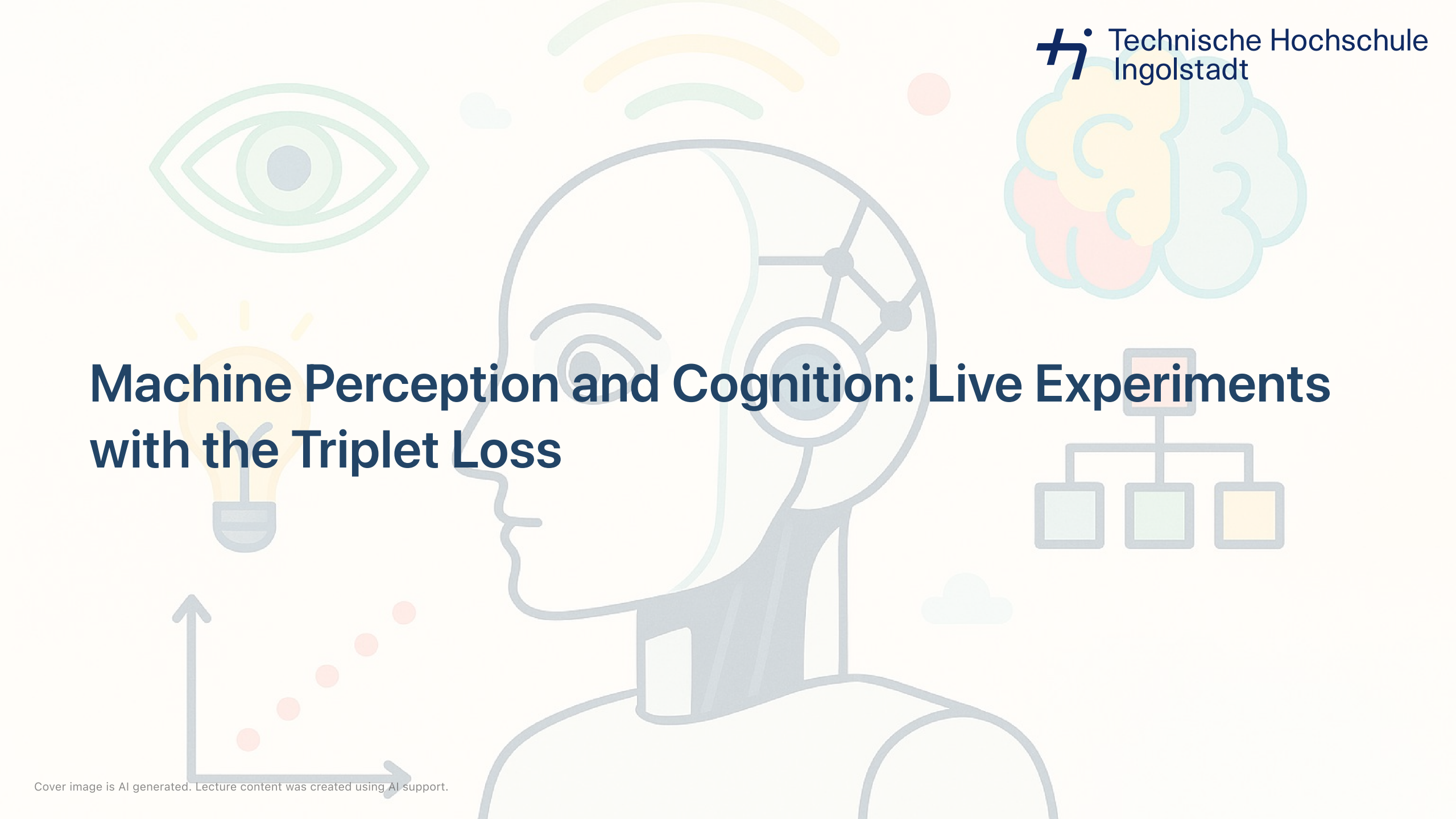
Triplet mining and modern losses:

- Most triplets are **easy** (loss already zero) — only **semi-hard** and **hard negatives** provide useful gradient
- **Online mining** selects informative triplets within each mini-batch from P classes × K samples

Face recognition application:

- Pipeline: detection → alignment → embedding → nearest-neighbor matching
- **Verification (1:1)**: threshold on distance; evaluated by FAR, FRR, and EER
- **Identification (1:N)**: nearest-neighbor search in gallery; new identities enrolled without retraining

Technische Hochschule Ingolstadt



Machine Perception and Cognition: Live Experiments with the Triplet Loss

Cover image is AI generated. Lecture content was created using AI support.

Live Experiment: Overview

Goal: Train a convolutional encoder with the **triplet loss** and evaluate it as a nearest-neighbour classifier on MNIST

Key ideas in this experiment:

- Prototype-based recognition via metric learning — no softmax classifier
- On-the-fly random triplet sampling with `TripletDataset`
- L2-normalised embeddings on the unit hypersphere
- `nn.TripletMarginLoss` from PyTorch
- Centroid-based 1-NN evaluation with scikit-learn
- OpenCV visualisation of query/prototype pairs

Task: Prototype-Based Digit Recognition

Why does standard classification fall short here?

A softmax classifier trained on digits 0–9 is **closed-set**: it can only output one of the 10 fixed classes and cannot enrol new classes without retraining.

Metric learning instead:

- Train an encoder f_θ so that $\| f_\theta(\mathbf{x}^a) - f_\theta(\mathbf{x}^p) \|_2$ is small for same-digit pairs and large for different-digit pairs
- At inference time, classify by **nearest-neighbour lookup** against class centroids built from a small set of prototype embeddings
- If the encoder generalises to a new class, it can be enrolled by embedding a few labelled examples — **no encoder retraining**

Configuration

```
N_EPOCHS                = 5     # training epochs
BATCH_SIZE              = 256   # triplet mini-batch size
LEARNING_RATE           = 1e-3  # AdamW learning rate
EMBED_DIM               = 64    # embedding space dimensionality
MARGIN                  = 0.5   # triplet loss margin α
N_PROTOTYPES_PER_DIGIT = 10    # prototype images held back per digit class
N_EVAL_SAMPLES          = 50    # test images queried against prototypes
N_SHOW                  = 20    # rows to print in the distance table
N_DISPLAY               = 3     # test images shown in the OpenCV window
SEED                    = 42    # random seed for reproducibility
```

Data Loading and Splitting

```
def load_mnist(root: str = "./data"):
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,)),  # MNIST mean / std
    ])
    train_dataset = torchvision.datasets.MNIST(
        root=root, train=True, download=True, transform=transform,
    )
    test_dataset = torchvision.datasets.MNIST(
        root=root, train=False, download=True, transform=transform,
    )
    return train_dataset, test_dataset
```

Prototype Split

```
def split_prototypes(dataset, n_prototypes_per_cls, rng):
    by_class = {c: [] for c in range(10)}
    for idx in range(len(dataset)):
        by_class[int(dataset.targets[idx])].append(idx)

    proto_indices, train_indices = [], []
    for cls, indices in by_class.items():
        rng.shuffle(indices)
        proto_indices.extend(indices[:n_prototypes_per_cls])
        train_indices.extend(indices[n_prototypes_per_cls:])
    return proto_indices, train_indices
```

`TripletDataset`: On-the-fly Sampling

```
class TripletDataset(torch.utils.data.Dataset):
    def __init__(self, dataset, indices):
        self.dataset  = dataset
        self.indices  = indices
        # Pre-compute per-class index lists for random sampling
        self.by_class = {}
        for idx in indices:
            label = int(dataset.targets[idx])
            self.by_class.setdefault(label, []).append(idx)
        self.classes = list(self.by_class.keys())

    def __len__(self):
        return len(self.indices)
```

`TripletDataset`: `__getitem__`

```
    def __getitem__(self, _):
        # Sample anchor class and two distinct indices within that class
        anc_cls          = random.choice(self.classes)
        anc_idx, pos_idx = random.sample(self.by_class[anc_cls], 2)

        # Sample negative from a different class
        neg_cls = random.choice([c for c in self.classes if c != anc_cls])
        neg_idx = random.choice(self.by_class[neg_cls])

        return self.dataset[anc_idx][0], \
               self.dataset[pos_idx][0], \
               self.dataset[neg_idx][0]
```



`TripletDataset`: `__getitem__`

The positional index `_` is **intentionally ignored** — every call produces a freshly sampled triplet. This is equivalent to **random triplet mining**: simple and sufficient for MNIST, but not optimal for harder datasets (recall semi-hard and batch-hard strategies from the theory chapter).

Image source: https://www.pexels.com/photo/light-bulb-on-black-background-20766488/

`ConvEmbedder`: Architecture

```
class ConvEmbedder(nn.Module):
    def __init__(self, embed_dim=64):
        super().__init__()
        self.conv_block = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),                  # (32, 14, 14)
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),                  # (64,  7,  7)
        )
        self.fc_block = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, 256),    # 3136 → 256
            nn.ReLU(inplace=True),
            nn.Linear(256, embed_dim),      # 256 → 64
        )
```

`ConvEmbedder`: L2 Normalisation

```
    def forward(self, x):
        features   = self.conv_block(x)
        embeddings = self.fc_block(features)
        return nn.functional.normalize(embeddings, p=2, dim=1)
```

`nn.functional.normalize` divides each row by $\max(\|\mathbf{z}\|_2, \varepsilon)$:

$$\hat{\mathbf{z}} = \frac{\mathbf{z}}{\max(\|\mathbf{z}\|_2, \varepsilon)}$$

`ConvEmbedder`: L2 Normalisation

Why normalise?

- All embeddings lie on the **unit hypersphere** $\mathcal{S}^{63}$ → distances are bounded in $[0, 2]$
- The margin $\alpha = 0.5$ is directly interpretable on this scale
- Euclidean distance and cosine similarity produce the **same nearest-neighbour ranking**: $\|\hat{\mathbf{u}} - \hat{\mathbf{v}}\|_2^2 = 2 - 2\hat{\mathbf{u}}^\top \hat{\mathbf{v}}$

Training: `nn.TripletMarginLoss`

```
optimizer = optim.AdamW(model.parameters(), lr=LEARNING_RATE)
criterion = nn.TripletMarginLoss(margin=MARGIN, p=2)
```

PyTorch's built-in loss implements the standard triplet objective:

$$\mathcal{L}(a, p, n) = \max(0,\ \|f_\theta(a) - f_\theta(p)\|_2 - \|f_\theta(a) - f_\theta(n)\|_2 + \alpha)$$

- `margin` (α): minimum required gap between positive and negative distance
- `p=2` : Euclidean (ℓ_2) distance
- Loss is **zero** when the constraint is already satisfied (easy triplet → no gradient)

AdamW adds decoupled weight decay to Adam; for metric learning, where embeddings are normalised, this is a mild regulariser.

Training Loop

```
for epoch in range(1, n_epochs + 1):
    model.train()
    total_loss, n_batches = 0.0, 0

    for anchors, positives, negatives in dataloader:
        anchors   = anchors.to(device)
        positives = positives.to(device)
        negatives = negatives.to(device)

        optimizer.zero_grad()

        emb_a = model(anchors)    # (batch, embed_dim)
        emb_p = model(positives)
        emb_n = model(negatives)

        loss = criterion(emb_a, emb_p, emb_n)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        n_batches  += 1
```

Prototype Embeddings and Centroids

```
def build_prototype_embeddings(model, dataset, proto_indices, device):
    embs, labels = embed_dataset_subset(model, dataset, proto_indices, device)

    proto_embs = np.zeros((10, embs.shape[1]), dtype=np.float32)
    for cls in range(10):
        cls_embs = embs[labels == cls]
        centroid = cls_embs.mean(axis=0)
        norm = np.linalg.norm(centroid)
        proto_embs[cls] = centroid / (norm + 1e-8)  # re-normalise

    return proto_embs, np.arange(10, dtype=np.int64)
```

1-NN Evaluation

```
def evaluate_knn(model, train_dataset, test_dataset,
                 proto_indices, n_eval, n_show, device):
    proto_embs, proto_labels = build_prototype_embeddings(
        model, train_dataset, proto_indices, device)

    test_embs, test_labels = embed_dataset_subset(
        model, test_dataset, eval_indices, device)

    nn_model = NearestNeighbors(n_neighbors=1, metric="euclidean")
    nn_model.fit(proto_embs)

    distances, nn_indices = nn_model.kneighbors(test_embs)
    pred_labels = proto_labels[nn_indices[:, 0]]
    accuracy    = np.mean(pred_labels == test_labels)
```

OpenCV Visualisation

```
def show_examples_with_prototypes(model, train_dataset, test_dataset,
                                  proto_indices, n_display, device, img_scale=4):
    # ...
    nn_model = NearestNeighbors(n_neighbors=3, metric="euclidean")
    nn_model.fit(proto_embs)
    # For each query: show [Query | Proto #1 | Proto #2 | Proto #3]
    cv2.imshow(window_title, full_img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Each 28 × 28 MNIST image is upscaled by `img_scale = 4` to 112 × 112 pixels and labelled with its digit class and distance. The **3 closest prototype images** (not centroids) are shown next to each query — making it easy to verify visually that the embedding space has learned meaningful geometry.

Experiments to Try

Modify `example_triplet_loss.py` and observe the effect:

Change	Expected observation
`MARGIN = 0.1`	Weaker constraint; clusters may overlap more
`MARGIN = 1.5`	Very strict; loss may remain high; convergence slower
`EMBED_DIM = 2`	Embeddings can be **plotted in 2-D**; visible digit clusters
`EMBED_DIM = 128`	Higher capacity; may generalise better with enough epochs
`N_PROTOTYPES_PER_DIGIT = 1`	One-shot setting: can the model still classify?

Recap!

Experiment: Metric learning with triplet loss on MNIST

- `TripletDataset` samples random (a, p, n) triplets on the fly — simple but effective for MNIST; harder datasets need mining strategies (semi-hard, batch-hard) from the theory chapter
- `ConvEmbedder` maps 28 × 28 images to 64-D L2-normalised embeddings on the unit hypersphere $\mathcal{S}^{63}$
- `nn.TripletMarginLoss` enforces $d_{ap} + \alpha \leq d_{an}$ and produces zero gradient for already-satisfied (easy) triplets

Recap!

Prototype gallery and evaluation:

- 10 prototype images per digit class are held back, embedded, and averaged into one centroid → Prototypical Network style
- 1-NN search with scikit-learn's `NearestNeighbors` against 10 centroids classifies new test images without retraining
- If the encoder generalises to a new class, a few labelled examples can be embedded and averaged into a new centroid — the encoder itself is never retrained

Recap!

Connection to the theory chapter:

- Triplet loss formulation and margin interpretation → *Triplet Loss* slides
- L2 normalisation and unit hypersphere → *Normalized Embeddings* slide
- Random vs. semi-hard vs. batch-hard mining → *Triplet Mining* slides
- Prototype-based 1-NN → *k-NN Classification with Embeddings* slide
- Prototype-based enrolment without encoder retraining → *Why Classification Falls Short* slide